CS & IT ENGINEERING

Operating System

Deadlock

Lecture -1

By- Vishvadeep Gothi sir

Recap of Previous Lecture

Topic Solution Without Busy Waiting

Topic Producer-Consumer Problem

Topic Reader-Writer Problem

Topics to be Covered

Topic Reader-Writer Problem

Topic Dining-Philosopher Problem

Topic : Reader-Writer Problem

Consider a situation where we have a file shared between many people:

- If one of the people tries editing the file, no other person should be reading or writing at the same time, otherwise changes will not be visible to him/her
- However, if some person is reading the file, then others may read it at the same time

Topic : Reader-Writer Problem Solution

- If writer is accessing the file, then all other readers and writers will be blocked
- If any reader is reading, then other readers can read but writer will be blocked

	Reader	writer
Reader	✓	X
writer	X	X

Topic : Reader-Writer Problem Solution

- Variables:
  - mutex: Binary Semaphore to provide Mutual Exclusion
  - wrt: Binary Semaphore to restrict readers and writers if writing is going on
  - Readcount: Integer variable, denotes number of active readers

- Initialization:
  - mutex: 1
  - w r t : 1
  - Read count: 0

or to block writters if reading is going on

Topic : Writer() & Reader() Processes

writer :-

```
wait(wrt)
  // performs writing
signal(wrt)
```

Reader :-

```
wait(mutex)
Readcount++;
if(Readcount==1)
  { wait(wrt); }
signal(mutex)
// performs reading
wait(mutex)
Readcount--;
if(Readcount==0)
  { signal(wrt); }
signal(mutex)
```

W 1 ⇒ writing
R 1 ⇒ stuck at wait(wrt)
R 2 ⇒ stuck at wait(mutex)

Topic : Dining Philosopher Problem

Topic : Dining Philosopher Problem

- K philosophers seated around a circular table
- There is one chopstick between each philosopher
- A philosopher may eat if he can pick up the two chopsticks adjacent to him
- One chopstick may be picked up by any one of its adjacent followers but not both

Topic : Dining Philosopher Problem Solution

Topic : Dining Philosopher Problem Solution

semaphore ch[k] = {1, 1, 1, ... ,1};

1 % 5 ⇒ 1
2 % 5 ⇒ 2
(4+1) % 5 ⇒ 0

Process for philosopher P_i

```
wait(ch[i])
wait(ch[(i+1)%k])

//eating

signal(ch[i])
signal(ch[(i+1)%k])
```

i = 0 ch[0] ch[1]

i = 1 ch[1] ch[2]

:

i = 4 ch[4] ch[0]

Topic : Dining Philosopher Problem Solution

suffers from deadlock

Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most (k–1) philosophers on the table

Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most (k–1) philosophers on the table
2. A philosopher should only be allowed to pick their chopstick if both are available at the same time

Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most (k–1) philosophers on the table
2. A philosopher should only be allowed to pick their chopstick if both are available at the same time
3. One philosopher should pick the left chopstick first and then right chopstick next; while all others will pick the right one first then left one

Topic : Operations on Resources

3 Operations on resources:

1. Request
2. Use
3. Release

granted

done

4. wait ⇒ when resource is not available

Topic : Deadlock

If two or more processes are waiting for such an event which is never going to occur

	Holds	waits
P1	keyboard	Harddisk
P2	Harddisk	printer
P3	printer	keyboard

Topic : Deadlock

Topic : Necessary Conditions for Deadlock

Deadlock can occur only when all following conditions are satisfied:

1. Mutual Exclusion ⇒ only one process can access a resource at a time.
2. Hold & Wait ⇒ all the deadlocked process must hold atleast one resource and must wait for atleast one resource
3. No-preemption → no any process is allowed to forcefully preempt resources from other processes.
4. Circular Wait

↓

all deadlocked processes should wait in circular manner

Topic : Resource Allocation Graph

Vertices

- Process 
- Resource 

Edges

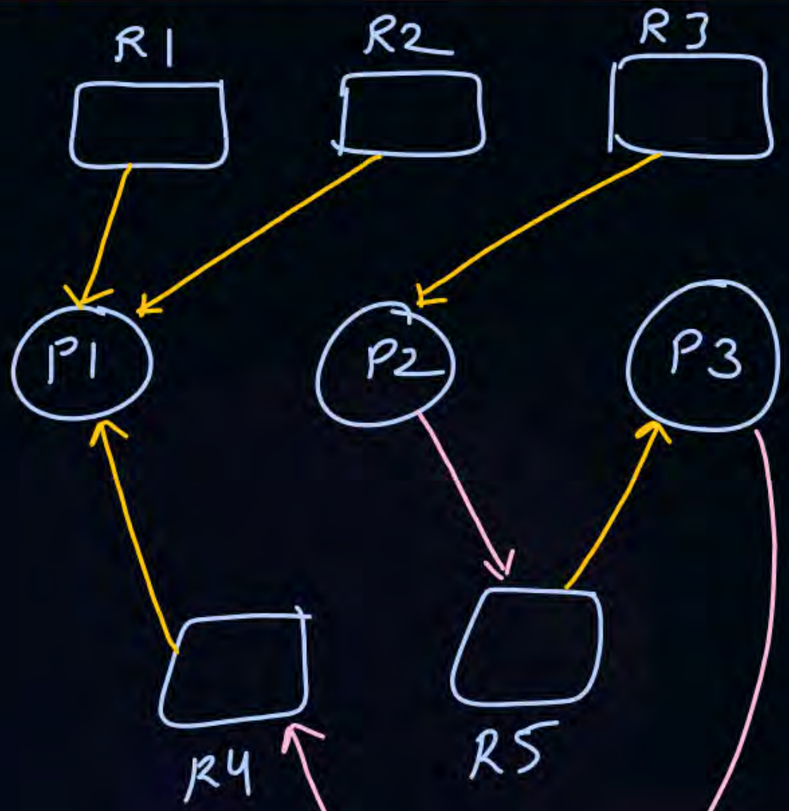
- Allocation Edge :- from resource (instance) to process

  denotes which resource is allocated to which process

- Request Edge :- from process to resource

  which process requested for which resource

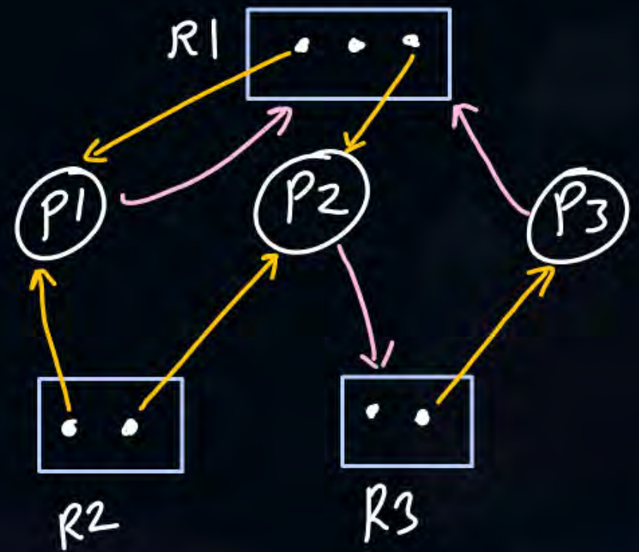
Topic : Resource Allocation Graph



when single instance of each resource

If multiple instances of resources are available

3 instances of resource R



2 mins Summary

Topic Reader-Writer Problem

Topic Dining-Philosopher Problem

Happy Learning

THANK - YOU